

```

In [ ]: import random
from collections.abc import Mapping

import marimo as mo
from ortools.sat.python import cp_model

def validar_n(n):
    if type(n) is not int or n < 1:
        raise ValueError("n tem de ser um inteiro positivo.")
    return n * n

class box:
    """Grupo genérico AllDifferent; coordenadas indexadas a partir de zero (R1).

    cells pode ser um dicionário {(linha, coluna): valor ou None} ou
    um iterável de pares (linha, coluna), inicialmente livres.
    """

    def __init__(self, cells=None, n=3):
        self.size = validar_n(n)
        self.n = n
        self.cells = {}
        if cells is not None:
            if isinstance(cells, Mapping):
                for (i, j), val in cells.items():
                    self.add(i, j, val)
            else:
                for i, j in cells:
                    self.add(i, j)

    def add(self, i, j, val=None):
        """Acrescenta ou atualiza uma célula; None deixa-a livre."""
        if any(type(c) is not int or not 0 <= c < self.size for c in (i, j)):
            raise ValueError(f"Coordenadas fora da grelha: {(i, j)}.")
        if val is not None and (type(val) is not int or not 1 <= val <= self.size):
            raise ValueError(f"0 valor deve estar entre 1 e {self.size}.")
        self.cells[i, j] = val
        return self

    def to_matrix(self):
        """Matriz nova; zero significa célula ausente ou livre."""
        matrix = [[0] * self.size for _ in range(self.size)]
        for (i, j), val in self.cells.items():
            if val is not None:
                matrix[i][j] = val
        return matrix

class cube(box):
    """Bloco n × n de índices (i, j), com 0 <= i, j < n (R2)."""

    def __init__(self, i, j, n=3):

```

```

        super().__init__(n=n)
        if any(type(c) is not int or not 0 <= c < n for c in (i, j)):
            raise ValueError("Índices de bloco fora do intervalo [0, n-1].")
        for row in range(i * n, (i + 1) * n):
            for col in range(j * n, (j + 1) * n):
                self.add(row, col)

class path(box):
    """Troço horizontal ou vertical, inclusivo e em qualquer sentido (R3)."""

    def __init__(self, inicio, fim, n=3):
        super().__init__(n=n)
        i0, j0 = inicio
        i1, j1 = fim
        # add valida ambos os extremos antes de construir o percurso.
        self.add(i0, j0)
        self.add(i1, j1)
        self.cells.clear()
        if i0 == i1:
            step = 1 if j1 >= j0 else -1
            for j in range(j0, j1 + step, step):
                self.add(i0, j)
        elif j0 == j1:
            step = 1 if i1 >= i0 else -1
            for i in range(i0, i1 + step, step):
                self.add(i, j0)
        else:
            raise ValueError("Um path tem de ser horizontal ou vertical.")

def pistas_aleatorias(n=3, k=None, seed=None):
    """Escolhe k posições distintas e valores aleatórios, sem garantir solução (R4)."""
    size = validar_n(n)
    if k is None:
        k = n
    if type(k) is not int or not 0 <= k <= size * size:
        raise ValueError(f"k tem de estar entre 0 e {size * size}.")
    rng = random.Random(seed)
    positions = rng.sample(range(size * size), k)
    clues = box(n=n)
    for position in positions:
        i, j = divmod(position, size)
        clues.add(i, j, rng.randint(1, size))
    return clues

class modelo_csp:
    """Uma variável inteira por célula e restrições obtidas apenas de grupos (R5)."""

    def __init__(self, n=3):
        self.size = validar_n(n)
        self.n = n
        self.model = cp_model.CpModel()
        self.variables = {

```

```

        (i, j): self.model.new_int_var(1, self.size, f"x_{i}_{j}")
        for i in range(self.size)
        for j in range(self.size)
    }
    self.status = None

def add(self, *groups):
    """Recebe zero ou mais grupos, sem distinguir a sua origem."""
    # Validar todos antes de acrescentar restrições ao modelo.
    snapshots = []
    for group in groups:
        if not isinstance(group, box) or group.n != self.n:
            raise ValueError("Cada grupo deve ser um box com o mesmo n
do modelo.")
        snapshots.append(box(group.cells, n=self.n))
    for group in snapshots:
        variables = [self.variables[cell] for cell in group.cells]
        if len(variables) > 1:
            self.model.add_all_different(variables)
        for cell, val in group.cells.items():
            if val is not None:
                self.model.add(self.variables[cell] == val)
    self.status = None
    return self

def solve(self):
    """Devolve uma matriz, ou None se for provado que não há
solução."""
    solver = cp_model.CpSolver()
    # Um trabalhador e uma seed fixa tornam os exemplos reprodutíveis.
    solver.parameters.num_search_workers = 1
    solver.parameters.random_seed = 0
    status = solver.solve(self.model)
    self.status = solver.status_name(status)
    if status == cp_model.INFEASIBLE:
        return None
    if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
        raise RuntimeError(f"O solver não concluiu a resolução:
{self.status}.")
    return [
        [solver.value(self.variables[i, j]) for j in range(self.size)]
        for i in range(self.size)
    ]

def montar_sudoku(n=3, pistas=None, extras=()):
    """Constrói linhas, colunas, blocos e um grupo de pistas (R6)."""
    size = validar_n(n)
    if pistas is None:
        pistas = pistas_aleatorias(n=n)
    groups = [path((i, 0), (i, size - 1), n=n) for i in range(size)]
    groups += [path((0, j), (size - 1, j), n=n) for j in range(size)]
    groups += [cube(i, j, n=n) for i in range(n) for j in range(n)]
    return modelo_csp(n).add(*groups, pistas, *extras)

def diagonais(n=3):
    """Bônus: X-Sudoku acrescentando dois box, sem alterar o modelo."""

```

```

size = validar_n(n)
return (
    box(((i, i) for i in range(size)), n=n),
    box(((i, size - 1 - i) for i in range(size)), n=n),
)

def validar_solucao(grelha, n=3, pistas=None, extras=()):
    """Valida independentemente do solver; levanta ValueError se houver erro."""
    size = validar_n(n)
    if grelha is None or len(grelha) != size or any(len(row) != size for row in grelha):
        raise ValueError("A solução não tem as dimensões esperadas.")
    if any(type(val) is not int for row in grelha for val in row):
        raise ValueError("As células da solução devem ser inteiros.")
    expected = set(range(1, size + 1))
    for i in range(size):
        if set(grelha[i]) != expected:
            raise ValueError(f"Linha {i} inválida.")
        if {grelha[j][i] for j in range(size)} != expected:
            raise ValueError(f"Coluna {i} inválida.")
    for i in range(n):
        for j in range(n):
            if {grelha[r][c] for r, c in cube(i, j, n=n).cells} != expected:
                raise ValueError(f"Bloco {(i, j)} inválido.")
    groups = list(extras)
    if pistas is not None:
        groups.append(pistas)
    for group in groups:
        if group.n != n:
            raise ValueError("Grupo de validação com dimensão incompatível.")
        values = [grelha[i][j] for i, j in group.cells]
        if len(values) != len(set(values)):
            raise ValueError("Um grupo viola a restrição todos diferentes.")
        for (i, j), val in group.cells.items():
            if val is not None and grelha[i][j] != val:
                raise ValueError(f"A pista {(i, j)} não foi preservada.")
    return True

def gerar_e_resolver(n=3, k=None, seed=27, max_tentativas=100):
    """Repete a geração aleatória se o CSP provar que as pistas são incompatíveis."""
    size = validar_n(n)
    if type(max_tentativas) is not int or max_tentativas < 1:
        raise ValueError("max_tentativas tem de ser um inteiro positivo.")
    # Um único box exige valores distintos também entre todas as pistas.
    if k is not None and type(k) is int and k > size:
        raise ValueError("Mais de n² pistas num só box nunca satisfazem AllDifferent.")
    rng = random.Random(seed)
    for attempt in range(1, max_tentativas + 1):
        clues = pistas_aleatorias(n=n, k=k, seed=rng.randrange(2**32))
        solution = montar_sudoku(n=n, pistas=clues).solve()

```

```

        if solution is not None:
            validar_solucão(solution, n=n, pistas=clues)
            return solution, clues, attempt
        raise RuntimeError(f"Não foram encontradas pistas solúveis em
{max_tentativas} tentativas.")

def mostrar_grelha(grelha, n=3):
    """Apresentação em texto, com separadores entre blocos; zero é um
ponto."""
    size = validar_n(n)
    width = len(str(size))
    separator = "+".join("-" * (n * (width + 1) - 1) for _ in range(n))
    lines = []
    for i, row in enumerate(grelha):
        if i and i % n == 0:
            lines.append(separator)
        chunks = [
            " ".join(f"{val if val else '.'}>{width}" for val in row[j:j +
n])
            for j in range(0, size, n)
        ]
        lines.append("|".join(chunks))
    return "\n".join(lines)

def executar_testes():
    """Testes dos requisitos, casos sem solução e generalidade dos
grupos."""
    def rejeita(function, *args, **kwargs):
        try:
            function(*args, **kwargs)
        except ValueError:
            return
        raise AssertionError("Era esperada uma exceção ValueError.")

    results = []
    for n in (3, 2):
        size = n * n
        group = box({(0, 0): 1, (1, 1): None}, n=n)
        matrix = group.to_matrix()
        assert len(matrix) == size and all(len(row) == size for row in
matrix)
        assert matrix[0][0] == 1 and sum(map(sum, matrix)) == 1
        matrix[0][0] = 0
        assert group.cells[0, 0] == 1 # A matriz não altera o grupo.
        for i, j, val in (
            (-1, 0, None), (size, 0, None), (0, -1, None), (0, size, None),
            (0, 0, 0), (0, 0, size + 1), (0.5, 0, None), (0, 0, True),
        ):
            rejeita(group.add, i, j, val)
        assert len(box(n=n).cells) == 0
        assert set(cube(n - 1, n - 1, n=n).cells) == {
            (i, j) for i in range(size - n, size) for j in range(size - n,
size)
        }
        rejeita(cube, -1, 0, n=n)
        rejeita(cube, n, 0, n=n)

```

```

        for start, end in (
            ((0, 0), (0, size - 1)), ((0, size - 1), (0, 0)),
            ((0, 0), (size - 1, 0)), ((size - 1, 0), (0, 0)),
        ):
            sequence = list(path(start, end, n=n).cells)
            assert len(sequence) == size and sequence[0] == start and
sequence[-1] == end
            assert list(path((1, 1), (1, 1), n=n).cells) == [(1, 1)]
            rejeita(path, (0, 0), (1, 1), n=n)
            rejeita(path, (0, 0), (0, size), n=n)
            assert len(pistas_aleatorias(n=n, k=0).cells) == 0
            assert len(pistas_aleatorias(n=n, k=size * size).cells) == size *
size
            assert pistas_aleatorias(n=n, seed=27).cells ==
pistas_aleatorias(n=n, seed=27).cells
            rejeita(pistas_aleatorias, n=n, k=-1)
            rejeita(pistas_aleatorias, n=n, k=size * size + 1)

# Fluxo obrigatório: gerar -> montar -> resolver -> validar.
            solution, clues, attempts = gerar_e_resolver(n=n, seed=27)
            assert validar_solucao(solution, n=n, pistas=clues)
            damaged = [row[:] for row in solution]
            damaged[0][0] = damaged[0][1]
            rejeita(validar_solucao, damaged, n=n, pistas=clues)
            changed_clues = box(clues.cells, n=n)
            (i, j), val = next(iter(changed_clues.cells.items()))
            changed_clues.add(i, j, val % size + 1)
            rejeita(validar_solucao, solution, n=n, pistas=changed_clues)

            incompatible = box({(0, 0): 1, (0, 1): 1}, n=n)
            unsatisfiable = montar_sudoku(n=n, pistas=incompatible)
            assert unsatisfiable.solve() is None and unsatisfiable.status ==
"INFEASIBLE"
            # A mesma célula pode aparecer em vários grupos, com pins
contraditórios.
            model = modelo_csp(n).add(box({(0, 0): 1}, n=n), box({(0, 0): 2},
n=n))
            assert model.solve() is None
            # Grupo arbitrário, sem geometria de linha/coluna/bloco.
            arbitrary = box({(0, 0): 1, (1, 2): 2, (2, 1): None}, n=n)
            grid = modelo_csp(n).add(arbitrary, box(n=n)).solve()
            assert grid[0][0] == 1 and grid[1][2] == 2
            assert len([grid[i][j] for i, j in arbitrary.cells]) == 3
            results.append((n, solution, clues, attempts))

        for invalid_n in (0, -1, 2.5, True):
            rejeita(box, n=invalid_n)
            rejeita(modelo_csp(2).add, box(n=3))
            rejeita(gerar_e_resolver, n=2, k=5)
            rejeita(gerar_e_resolver, max_tentativas=0)
            # Extensão diagonal validada com o mesmo modelo CSP.
            extra_groups = diagonais(n=2)
            diagonal_solution = montar_sudoku(n=2, pistas=box(n=2),
extras=extra_groups).solve()
            assert validar_solucao(diagonal_solution, n=2, extras=extra_groups)

```

```
assert gerar_e_resolver(n=1)[0] == [[1]]
return results
```

Sudoku genérico como CSP — solução

R1-R3: `box(cells=None, n=3)` representa qualquer conjunto de células através de um dicionário `(linha, coluna) → valor ou None`. As coordenadas começam em zero. `add` valida coordenadas e valores; `to_matrix` produz a matriz de apresentação. `cube(i, j, n=3)` e `path(inicio, fim, n=3)` apenas escolhem as células do grupo.

R4: `pistas_aleatorias(n=3, k=None, seed=None)` escolhe `k` posições distintas, por omissão `k=n`, e atribui valores aleatórios entre 1 e n^2 .

R5-R6: `modelo_csp(n=3)` cria uma variável inteira por célula. `add(*groups)` aplica `AllDifferent` e fixa os valores de cada grupo. `montar_sudoku` reúne linhas, colunas, blocos e pistas. `solve()` devolve uma matriz ou `None` quando não há solução.

Modelação e escolhas

Para cada célula usamos $x_{ij} \in \{1, \dots, n^2\}$. Para cada grupo G , impomos $\text{AllDifferent}(x_{ij} : (i, j) \in G)$ e, se uma célula tem pista v , impomos $x_{ij} = v$. Como cada linha, coluna e bloco contém n^2 células, os valores distintos no domínio indicado são necessariamente todos os valores de 1 a n^2 .

Escolhi **CP-SAT do OR-Tools** porque suporta variáveis inteiras e `AllDifferent` diretamente, propagando restrições durante a pesquisa. O modelo desconhece a geometria dos grupos: a lógica de restrições é única. O dicionário permite guardar apenas as células pertencentes ao grupo e impede coordenadas duplicadas dentro do mesmo grupo.

Pistas aleatórias podem ser incompatíveis. Conforme o enunciado, as pistas também constituem um grupo `AllDifferent`: duas pistas com o mesmo valor tornam esse grupo impossível, mesmo em células afastadas. Por isso, mais de n^2 pistas num único grupo nunca têm solução. `pistas_aleatorias` conserva esta geração sem filtrar os valores; `gerar_e_resolver` tenta novas pistas até obter uma solução, com um limite explícito de 100 tentativas. O solver distingue impossibilidade provada (`None`) de outros estados não conclusivos (exceção).

A solução encontrada **não garante unicidade**: poucas pistas podem admitir várias soluções, e o exercício pede apenas uma grelha válida.

```
In [ ]: resultados = executar_testes()
mo.md("**Testes automáticos concluídos:** R1–R6, grelhas 9×9 e 4×4, "
      "percursos nos dois sentidos, entradas inválidas, pistas preservadas, "
      "grupos arbitrários, CSP sem solução e extensão diagonal.")
```

Testes automáticos concluídos: R1–R6, grelhas 9×9 e 4×4, percursos nos dois sentidos, entradas inválidas, pistas preservadas, grupos arbitrários, CSP sem solução e extensão diagonal.

```
In [ ]: mo.vstack([
    mo.md(
        f"### n={n}: Sudoku {n*n}x{n*n}\n\n"
        f"{len(clues.cells)} pistas aleatórias; {attempts}
tentativa(s).\n\n"
        f"Pistas (`. ` = célula livre):\n\n``text\n"
        f"{mostrar_grelha(clues.to_matrix(), n=n)}\n\n``\n\n"
        f"Solução validada:\n\n``text\n{mostrar_grelha(solution,
n=n)}\n\n``"
    )
    for n, solution, clues, attempts in resultados
])
```

n=3: Sudoku 9×9

3 pistas aleatórias; 1 tentativa(s). Pistas ( = célula livre):

```
. . .|. . .|. . .  
. . .|. . .|. . .  
. . .|. . 1|. . .  
-----+-----+-----  
. . .|. . .|. . 6  
. . .|. . .|. . .  
. . .|. . .|. . .  
-----+-----+-----  
. . .|. . .|. . .  
. . .|5 . .|. . .  
. . .|. . .|. . .
```

Solução validada:

```
2 3 6|9 8 4|1 5 7  
4 1 5|3 7 6|2 8 9  
7 9 8|2 5 1|6 3 4  
-----+-----+-----  
3 5 4|1 2 7|8 9 6  
1 6 2|8 3 9|4 7 5  
8 7 9|4 6 5|3 1 2  
-----+-----+-----  
5 4 3|7 1 2|9 6 8  
6 2 1|5 9 8|7 4 3  
9 8 7|6 4 3|5 2 1
```

n=2: Sudoku 4×4

2 pistas aleatórias; 1 tentativa(s). Pistas ( = célula livre):

```
. .|. .  
1 3|. .  
---+---  
. .|. .  
. .|. .
```

Solução validada:

```
4 2|3 1  
1 3|2 4  
---+---  
2 1|4 3  
3 4|1 2
```

Extensão e utilização

O bónus diagonal usa `diagonais(n)` para construir dois `box` livres e passa-os a `montar_sudoku(..., extras=diagonais(n))`. Regiões irregulares ou blocos sobrepostos podem, da mesma forma, ser descritos por novos `box`.

Para experimentar outra dimensão ou outro conjunto aleatório de pistas:

```
grelha, pistas, tentativas = gerar_e_resolver(n=3, k=3, seed=42)
validar_solucao(grelha, n=3, pistas=pistas)
print(mostrar_grelha(grelha, n=3))
```

Aumentar `n` aumenta o número de variáveis para n^4 ; não foi realizado o bónus de desempenho 36×36 nem a extensão tridimensional.